Approximation Algorithms

Ben Heinze

April 16, 2026

Contents

1 NP-Complete **1**

2 Approximation Algorithms **1**
2.1 Simple Example: . . . 2
2.2 Vertex Cover . . . 2
2.2.1 Hierarchies: . . . 2
2.3 Set Cover . . . 3
2.3.1 Solving Set Cover . . . 3

3 Knapsack problem **4**
3.1 DP method . . . 4
3.1.1 Values instead of weights . . . 5

1 NP-Complete

Techniques to hand NP Compelte

- Brute force (ie exponential time)
- Heuristics
- approximation algorithms
- Fixed parameter tractable algorithms

2 Approximation Algorithms

$ALG \leq \alpha OPT$

- AGL: cost (size) of algorithms solution
- α: approximation ratio
- OPT: cost (size) of optimal solution

If problem is a maximization problem, $ALG \geq \frac{1}{\alpha} OPT$

2.1 Simple Example:

Suppose I know my algorithm is a 1.12-approximation algorithm ($\alpha = 1.12$). Suppose my algorithm returns a solution of cost=746.125. Then, we know that $756.125 \leq 1.12OPT$.

$\implies \frac{746.125}{1.12} = 666.183 \leq OPT \leq 746.125$

The will give us a guaranteed upper and lower bound of where OPT should be. Then, ask yourself if the known solution is good enough for your purposes.

2.2 Vertex Cover

VC: given graph, find smallest subset of vertices such that everye edge in the graph has at least one vertex in the subset.

```
ALG Vertex Cover Pseudo-code:

While uncoverd edge exists
  select both of its vertices
```

Consider a set of edges $E' \subset E$, that do not share vertices. Is there a relationship between the minimum vertex cover and $|E'|$?

$|E'| \leq OPT$, where OPT is the size of actual smallest vertex cover. If we slected fewer than one vertex per edge, we would not have a vertex cover because that edge would not be covered.

Does the size of the algorithm's output relate to a set of edges that do not share verticies? The algorithm finds a specific subset E' since the algorithm is constructed to only pick both of the uncovered edge's vertices. So if we have 10 edges, then the most the algorithm covers is 10*2 (since it selects two vertices for edge edge): $ALG = 2|E'|$.

$$\implies ALG = 2|E'| \leq 2OPT \implies ALG \leq 2OPT$$

To write an approximiation algorithm, the goal is to relate the optimal solution to some structure S, then relating the algorithm to S. We cannot find optimal vertex covers in poly time unless $P = NP$, but this algorithm is at worst 2-times optimal. Two ways to get a better approximation:

1. Find a new algorithm that performs better
2. Analyze the already-existing algorithm and decrease the $ALG \leq OPT$ bounds

2.2.1 Hierarchies:

Show the Computibility Hierarchy

Approximability Hierarchy

- NPO Set
  - APX subset in NPO
    - Vertex Cover is an item in APX
    - PTAS within APX
      - FPTAS subset within PTAS
      - Knapsack problem within FPTAS
      - PO subset within mystery subset (polynomial optimization)

        - shorest path
    - Set cover is an item of NPO (other people proved this, we will trust ti)

APX: optimization problems that can be approximated within a constant ratio

2.3 Set Cover

Set Cover: Given a set of elements (universe), and sets containing those elements, find the smallest number of sets so that every element of the universe is included.

$U = \{1, 4, 7, 8, 10\}$
$S = \{\{1, 7, 8\}, \{1, 4, 7\}, \{8, 4, 7\}, \{4, 8, 10\}\}$

```
Basic Set Cover Greedy Algorithm:

While element of universe is not included
  select S_i with largest number of excluded elements
```

2.3.1 Solving Set Cover

Suppose the universe contains n elements:

$$ALG \leq \alpha OPT$$

- ALG: # of sets selected by the algortihm to cover all n elements
- OPT: # of sets inn an optimal solution cover all n elements

Game plan: bound max number of sets in $ALG\ldots$ by bounding the max number of iterations of the algorithm... by bounding the size of each set added by the algorithm.

For the first set, we want to select the largest set since it covers the most unique elements (since we dont have any elements yet).

$$? \leq |\text{Biggest Set}| \leq ?$$

We are guaranteed to do at least as good as lower bound, guaranteed we don't do better than upper bound. Think about this. In this case, we want to find the relationship between the lower bound. Once we have this relationship, everything else falls into place

Suppose the universe contains n elements. What wcan we say about the first set selected? Answer: It's the biggest!

$$\text{claim: } \frac{n}{OPT} \leq |\text{Biggest Set}|$$

proof:

$$\text{suppose } \frac{n}{OPT} > |\text{Biggest Set}| \leq |Everyset|$$

How many sets does the optimal solution S^{OPT} use: OPT. Then how many elements are covered by those OPT sets?

Suppose the universe contains n elements. The first set selected will have $\geq \frac{n}{OPT}$ elements.

Then, the number of elements remaining after the first iteration is *original # of elements - # elements removed* $\rightarrow n \geq \frac{n}{OPT}$

$$n_1 \leq n - \frac{n}{OPT} = n\left(1 - \frac{1}{OPT}\right)$$

What can we sayabout the second set selected? **It covers the most uncovered elements**.

- Suppose the first set *was* in the optimal solution
  - Then, a remaining set must have at least $\frac{n_1}{OPT-1}$ uncovered elements.

$$\implies \frac{n_1}{OPT} < \frac{n_1}{OPT-1} \leq |SecondSet|$$

- Suppose the first set *was not* in the optimal solution
  - If not, how do the remaining $OPT - 1$ optimal sets cover the remaining n_1 elements?

$$\implies \frac{n_1}{OPT} \leq |SecondSet|$$

In general, after t iteration:

$$n_t \leq n_{t-1} - \frac{n_{t-1}}{OPT} = n_{t-1}\left(1 - \frac{1}{OPT}\right) \leq \ldots \leq n\left(1 - \frac{1}{OPT}\right)^t$$

The question is what is t? When we solve t, we solve ALG.
Big picture:

1. How many sets are added each iteration: 1
2. $ALG = \#$ of iterations
3. When does the algorithm terminate: $n_t < 1$

What value of t makes $n_t < 1$:

$$\implies ALG \leq ln(n)OPT$$

It turns out that Set Cover cannot be approximated within the bound of $(1 - o(1))ln(n)$, unless P = NP

3 Knapsack problem

Knapsack: given a set of n items with values $v_1, \ldots, v_n$ and weights $w_1, \ldots, w_n$, select the most valuable combination with total weight $\leq W$

3.1 DP method

How could we approach this? **Dynamic programming sense?**

Algorithm: is there an optimal substructure? I.E., if I have an optimal solution to some instance, does that imply an optimal solution to a different instance?

***He's going to talk about the intuitive DP version of the problem, then explain why it wouldn't work. ***

$$\text{Define } opt(i, w) = max\left(opt(i-1, w), opt(i-1, w - w_i) + v_1\right)$$

What now:

- for every single weight $\leq$ W and i, calculate $opt(i, w)$
- no longer polynomial
- reducing search space of weights will compromise weight accuracy. not allowed!

If you make a table with items as the rows and total weights as the columns for the DP, how can you check every possible weight? Its trying to turn something continuous into something discrete, and adding different increments of weights will be difficult and it breaks the DP.

Instead, we need to compromise value accuracy isntead of weights.

3.1.1 Values instead of weights

Instead of saying "for a given weight, how much value can we get?" to "given a value threshold, whats the smallest weight i can get?"

Now we have a table with items as the rows and total value as the columns. For every possible total backpack value, go to the bottom row and check each column until the weight threshold is crossed, then we just go back one spot. That will lead to the optimal solution.

Define $opt(i, V)$ = minimum weight of subset of items $\{1, \ldots, i\}$ that gives value at least V (target value).

Case 1: if i is not in the optimal solution for $\{1, \ldots, i\}$:

- $opt(i, V) = opt(i - 1, V)$

Case 2: if i is in the optimal solution for $\{1, \ldots, i\}$:

- $opt(i, V) = opt(i - 1, V - v_i) + w_i$

We are going to assume the values are integers

Algorithim:

1. For $0 \leq i \leq n$ and $0 \leq V \leq n$ max(i under max idk how to type that) v_i, compute $opt(i, V)$.
2. Return $V* = max\{V : opt(i, V) \leq W\}$

Running time: $O(n^2$max (i under max)$v_i)$

To make the running time better, we can change the units of the values of the table to lower values, which will increase, but it's not easy for compliated numbers and you can lose information. See below, how 500000 keeps all the information, but the 163882 loses 882 dollors of accuracy:

$$500000 \text{ dollars } \rightarrow 5 \text{ hundred-thousand dollars}$$

$$163882 \text{ dollars } \rightarrow 160 \text{ hundred-thousand dollars}$$

Algorithm:

1. Let v_{max} = max i under maxv_i
2. For each i, let $v'_i = \lfloor v_i \frac{n}{\epsilon v_{max}} \rfloor$
3. Run dynamic programming algorithm using $\{v'_i\}, \{w_i\}, W$

Running Time = $O\left(\frac{n^3}{\epsilon}\right)$

How does scaling values impact the cost? (how does ALG relate to OPT)

$\$S_{\text{ALG}}$ = set the algorithm selected. $\$S_{\text{OPT}}$ = set the optimal items.

Need to relate $\sum_{i \in S_{OPT}} v_i$ relate to $\sum_{i \in S_{ALG}} v_i$

But the algorithm operates on v_i':
Need to relate $\sum_{i \in S_{OPT}} v_i$ to $\sum_{i \in S_{OPT}} v_i'$ to $\sum_{i \in S_{ALG}} v_i'$ to $\sum_{i \in S_{ALG}} v_i$
$\sum_{i \in S_{OPT}} v_i' = \lfloor v_i \frac{n}{\epsilon v_{max}} \rfloor$
He does algebra:
$\sum_{i \in S_{OPT}} v_i' \leq OPT \frac{n}{\epsilon v_{max}} - n$
How does $ALG = \sum_{i \in S_{ALG}} v_i$ to $OPT = \sum_{i \in S_{OPT}} v_i$?
He does algebra:
$\sum_{i \in S_{OPT}} v_i' \leq OPT \frac{n}{\epsilon v_{max}} - n$

ALG is optimal on the scaled values: ALG is the beest at identifying the **highest *scaled* amount** of value while OPT is the best at identifying the **highest amount** of value in the backpack.

We test OPT on how well it can perform on the scaled values, and it performs worse than ALG because it's playing a game it wasn't designed to do; OPT is not OPT on the scaled values, but ALG *is*.

Im not writing all this math. look at the slides if you want. (end of lecture 4/16).

We relate OPT to ϵv_{max}.

Performance guaranteee: $ALG \geq (1 - \epsilon)OPT$

Running time: $O\left(\frac{n^3}{\epsilon}\right)$

This is called a fully polynomial-time approximation scheme (FPTAS)

We can solve Knapsack instances arbitrarily close to optimal in polynomial time!